

Embedded Systems Homework #6

Nidhi Borhade
StudentId : 109998404

October 2023

Contents

1	Network Domains	2
2	Routing	3
3	Linux Driver device	4
4	Extension for I/O Support	11

1 Network Domains

```
1 192.168.  0.  0
2 255.255.255.240
3 -----
4 192.168.  0.  0 -> ID
5 192.168.  0. 15 -> Broadcast Adress
6
7 14 Host adresses left in range of 192.168.0.1 -
  192.168.0.14
```

2 Routing

1. How to observe the routing information (route table) on the Linux host?

```
1 netstate -rn
2
3 1) n - shows numeric port numbers and ip
   addresses
4 2) r - shows routing table
```

What do the SYN and ACK flags on TCP packets mean?

```
1 SYN Flag - The synchronization flag is used to
   establish a three-way handshake between
   two hosts. Only the first packet from both
   the sender and receiver should have this
   flag set.
2 ACK Flag - will be sent with ID, to make sure
   every package arrived well
```

3 Linux Driver device

Write a basic Linux device driver program which explains about the kernel thread in Linux device driver.

```
1 #include <linux/kernel.h>           // Includes
   kernel-related functions and macros.
2 #include <linux/init.h>             // Includes
   module initialization and exit macros.
3 #include <linux/module.h>           // Includes
   module-related macros and data structures.
4 #include <linux/kdev_t.h>           // Includes
   kernel device number types and macros.
5 #include <linux/fs.h>               // Includes
   file system-related structures and functions
   .
6 #include <linux/cdev.h>             // Includes
   character device-related structures and
   functions.
7 #include <linux/device.h>           // Includes
   device model structures and functions.
8 #include <linux/slab.h>             // Includes
   memory allocation functions like kmalloc().
9 #include <linux/uaccess.h>          // Includes
   functions for user space data access.
10 #include <linux/kthread.h>          // Includes
   kernel thread management functions and data
   structures.
11 #include <linux/sched.h>            // Includes
   task management functions and task_struct
   structure.
12 #include <linux/delay.h>            // Includes
   delay functions like msleep().
13 #include <linux/err.h>              // Includes
   error code definitions and error handling
   functions.
14
```

```

15 dev_t dev = 0;                                // Device
    numbers.
16 static struct class *dev_class;                // Device
    class.
17 static struct cdev etx_cdev;                    //
    Character device structure.
18
19 static int __init etx_driver_init(void); //
    Function for module initialization.
20 static void __exit etx_driver_exit(void); //
    Function for module exit.
21
22 static struct task_struct *etx_thread;          //
    Kernel thread structure.
23
24 /*
25 ** Function Prototypes
26 */
27 /***** Driver functions
    *****/
28 static int etx_open(struct inode *inode, struct
    file *file); // Function for opening the
    device.
29 static int etx_release(struct inode *inode,
    struct file *file); // Function for closing
    the device.
30 static ssize_t etx_read(struct file *filp, char
    __user *buf, size_t len, loff_t * off); //
    Function for reading from the device.
31 static ssize_t etx_write(struct file *filp,
    const char *buf, size_t len, loff_t * off);
    // Function for writing to the device.
32 /*****
33
34 int thread_function(void *pv); // Function for
    the kernel thread.
35

```

```

36  /*
37  ** Thread
38  */
39  int thread_function(void *pv)
40  {
41      int i=0;
42      while(!kthread_should_stop()) { // Run the
          thread until it's told to stop.
43          pr_info("In EmbeTronicX Thread Function
              %d\n", i++); // Log a message.
44          msleep(1000); // Sleep for 1000
              milliseconds.
45      }
46      return 0;
47  }
48
49  /*
50  ** File operation structure
51  */
52  static struct file_operations fops =
53  {
54      .owner          = THIS_MODULE,
55      .read            = etx_read,
56      .write           = etx_write,
57      .open            = etx_open,
58      .release         = etx_release,
59  };
60
61  /*
62  ** This function will be called when we open the
        Device file
63  */
64  static int etx_open(struct inode *inode, struct
        file *file)
65  {
66      pr_info("Device File Opened...!!!\n");
          // Log a message when the device is
          opened.

```

```

67         return 0;
68     }
69
70     /*
71     ** This function will be called when we close
       the Device file
72     */
73     static int etx_release(struct inode *inode,
       struct file *file)
74     {
75         pr_info("Device File Closed...!!!\n");
       // Log a message when the device is
       closed.
76         return 0;
77     }
78
79     /*
80     ** This function will be called when we read the
       Device file
81     */
82     static ssize_t etx_read(struct file *filp,
83         char __user *buf, size_t len,
84         loff_t *off)
85     {
86         pr_info("Read function\n"); // Log a
       message when reading from the device.
87
88         return 0;
89     }
90
91     /*
92     ** This function will be called when we write
       the Device file
93     */
94     static ssize_t etx_write(struct file *filp,
95         const char __user *buf, size_t
       len, loff_t *off)

```

```

96         pr_info("Write Function\n"); // Log a
          message when writing to the device.
97         return len;
98     }
99
100     /*
101     ** Module Init function
102     */
103     static int __init etx_driver_init(void)
104     {
105         /*Allocating Major number*/
106         if((alloc_chrdev_region(&dev, 0, 1, "
          etx_Dev")) < 0){ // Allocate a major
          number for the device.
107             pr_err("Cannot allocate major
              number\n"); // Log an error
              if allocation fails.
108             return -1;
109         }
110         pr_info("Major = %d Minor = %d \n",MAJOR
          (dev), MINOR(dev)); // Log the major
          and minor numbers.
111
112         /*Creating cdev structure*/
113         cdev_init(&etx_cdev, &fops); //
          Initialize the character device
          structure with file operations.
114
115         /*Adding character device to the system
          */
116         if((cdev_add(&etx_cdev, dev, 1)) < 0){
          // Add the character device to the
          system.
117             pr_err("Cannot add the device to the
              system\n"); // Log an error if
              addition fails.
118             goto r_class;
119         }

```

```

120
121      /*Creating struct class*/
122      if(IS_ERR(dev_class = class_create(
          THIS_MODULE, "etx_class"))){ //
          Create a device class.
123          pr_err("Cannot create the struct
              class\n"); // Log an error if
              creation fails.
124          goto r_class;
125      }
126
127      /*Creating device*/
128      if(IS_ERR(device_create(dev_class, NULL,
          dev, NULL, "etx_device"))){ //
          Create the device.
129          pr_err("Cannot create the Device \n
              "); // Log an error if creation
              fails.
130          goto r_device;
131      }
132
133      etx_thread = kthread_create(
          thread_function, NULL, "eTx Thread");
          // Create a kernel thread.
134      if(etx_thread) {
135          wake_up_process(etx_thread); //
          Start the kernel thread.
136      } else {
137          pr_err("Cannot create kthread\n");
          // Log an error if thread
          creation fails.
138          goto r_device;
139      }
140
141      pr_info("Device Driver Insert...Done!!!\n
          n"); // Log a success message.
142      return 0;
143

```

```

144
145 r_device:
146     class_destroy(dev_class); // Clean up
        class if there's an error.
147 r_class:
148     unregister_chrdev_region(dev, 1); //
        Unregister the allocated major/minor
        numbers.
149     cdev_del(&etx_cdev); // Delete the
        character device.
150     return -1;
151 }
152
153 /*
154 ** Module exit function
155 */
156 static void __exit etx_driver_exit(void)
157 {
158     kthread_stop(etx_thread); // Stop the
        kernel thread.
159     device_destroy(dev_class, dev); //
        Destroy the device.
160     class_destroy(dev_class); // Destroy
        the device class.
161     cdev_del(&etx

```

4 Extension for I/O Support

```
1      /* Add new data buffer and related variables */
2  #define BUF_SIZE 1024
3  static char device_buffer[BUF_SIZE]; // Data buffer
4      to store information to be read or written.
5
6  /* ... Other code ... */
7
8  /*
9  ** This function will be called when we read the
10     Device file
11  */
12  static ssize_t etx_read(struct file *filp, char
13     __user *buf, size_t len, loff_t *off)
14  {
15     int bytes_to_copy, remaining_bytes;
16
17     if (*off >= BUF_SIZE) {
18         /* End of file */
19         return 0;
20     }
21
22     remaining_bytes = BUF_SIZE - *off;
23     bytes_to_copy = min(remaining_bytes, len);
24
25     if (copy_to_user(buf, device_buffer + *off,
26         bytes_to_copy) != 0) {
27         pr_err("Data copy to user failed\n");
28         return -EFAULT; // Error code for failed
29             memory copy.
30     }
31
32     *off += bytes_to_copy;
33     return bytes_to_copy;
34 }
```

```

31  /*
32  ** This function will be called when we write the
    Device file
33  */
34  static ssize_t etx_write(struct file *filp, const
    char __user *buf, size_t len, loff_t *off)
35  {
36      int bytes_to_copy, remaining_bytes;
37
38      if (*off >= BUF_SIZE) {
39          /* End of file */
40          return -ENOSPC; // Error code for no space
                left.
41      }
42
43      remaining_bytes = BUF_SIZE - *off;
44      bytes_to_copy = min(remaining_bytes, len);
45
46      if (copy_from_user(device_buffer + *off, buf,
        bytes_to_copy) != 0) {
47          pr_err("Data copy from user failed\n");
48          return -EFAULT; // Error code for failed
                memory copy.
49      }
50
51      *off += bytes_to_copy;
52      return bytes_to_copy;
53  }

```